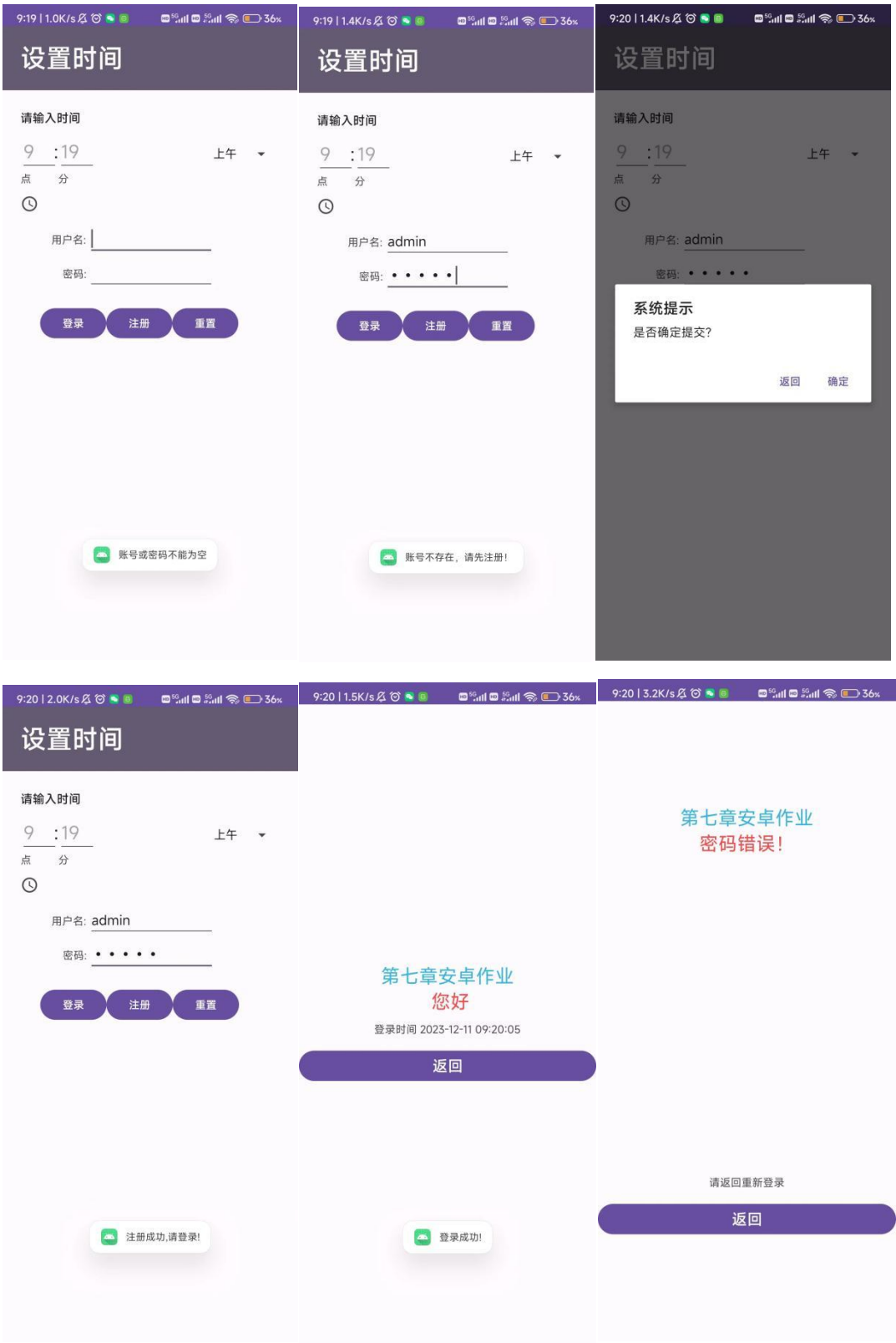


第七章：
p213 7.2 使用 SQLite 实现记录登录时间的功能，并且能够查询登录的历史记录。
1. 运行界面：



2. 部分代码展示:

```
public class MainActivity extends AppCompatActivity implements  
View.OnClickListener {
```

```
    //变量声明
```

```
    SQLiteDatabase sqLiteDatabase;
```

```
    DataBaseHelper helper;
```

```
    private EditText username;
```

```
    private EditText password;
```

```
    private String name_str;
```

```
    private String paswd_str;
```

```
    private String time_str;
```

```
    private Button resign;
```

```
    private Button login;
```

```
    private Button btn_new;
```

```
    private String mstr="";
```

```
    private final static String TAG="MainActivity";
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
```

```
        //控件定位
```

```
        helper=new DataBaseHelper(this); //传递上下文
```

```
        username=(EditText) findViewById(R.id.username);
```

```
        password=(EditText) findViewById(R.id.password);
```

```
        resign=(Button) findViewById(R.id.resign);
```

```
        login=(Button) findViewById(R.id.login);
```

```
        btn_new=(Button)findViewById(R.id.btn_new);
```

//①以读写的方式打开数据库，一旦数据库的磁盘空间满了，数据库就只能读而不能写，倘若使用的是 `getWritableDatabase()` 方法就会出错

//②还有一个 `getReadableDatabase()`方法也是以读写方式打开数据库，如果数据库的磁盘空间满了，就会打开失败，当打开失败后会继续尝试

//以只读方式打开数据库。如果该问题成功解决，则只读数据库对象就会关闭，然后返回一个可读写的数据库对象。

```
sqLiteDatabase = helper.getWritableDatabase();
```

```
//设置按钮监听
```

```
resign.setOnClickListener(this);
```

```
login.setOnClickListener(this);
```

```
btn_new.setOnClickListener(this);
```

```
}
```

```
@Override
```

```
public void onClick(View v) {
```

```
    int a = 0;
```

```
    if (v.getId() == R.id.resign)
```

```

        a = 1;
    else if (v.getId() == R.id.login)
        a = 2;
    else if (v.getId() == R.id.btn_new)
        a = 3;
    // 通过 id 来分配按钮事件
    if (a == 1) {
        //注册相关的弹窗设定，如果确定提交，则完成注册，数据存入数据库
        //AlertDialog: 对话框控件
        new AlertDialog.Builder(MainActivity.this).setTitle("系统提示")
            .setMessage("是否确定提交? ")
            //为确定按钮配置监听
            .setPositiveButton("确定", new
DialogInterface.OnClickListener() {
                @Override
                public void onClick(DialogInterface arg0, int arg1) {
                    // 获取 EditText 中的内容，并将其转化成字符串型后存入变量中
                    name_str = username.getText().toString();
                    paswd_str = password.getText().toString();
                    time_str = DateUtil.getNowTimeDetail(); //给数据
表设置游标，Cursor 是一个游标，以 name 字段为依据，query 是一种根据条件
获取数据的方法
                    Cursor cursor = sqLiteDatabase.query(Constants.TABLE_NAME, new
String[]{"name"}, "name=?", new String[]{name_str}, null, null, null);
                    //如果游标找到了所需要的 name，则返回已注册，否则就利用之前写的 insert 方
法插入数据
                    if (cursor.getCount() != 0) {
                        Toast.makeText(MainActivity.this, "该用户已
注册!", Toast.LENGTH_SHORT).show();
                    } else {
                        helper.insert(sqLiteDatabase, name_str,
paswd_str, time_str);
                        Toast.makeText(MainActivity.this, "注册成功,
请登录!", Toast.LENGTH_SHORT).show();
                    }
                }
            }).setNegativeButton("返回", new
DialogInterface.OnClickListener() {
                public void onClick(DialogInterface arg0, int arg1) {
                    //这个是点击返回后的操作，因为不需要，所以
不管他直接跳出就好。
                }
            })

```

```

        }).show();
    }
    else if (a == 2) {
        //登录按钮功能，上面解释过的已省略
        String user_str = username.getText().toString();
        String psw_str = password.getText().toString();
        //账号或密码为空时
        if (user_str.equals("")) {
            Toast.makeText(this, "账号或密码不能为空",
Toast.LENGTH_SHORT).show();
        } else {
            Cursor cursor = sqLiteDatabase.query(Constants.TABLE_NAME,
new String[]{"password"}, "name=?", new String[]{user_str}, null, null, null);
            //游标的遍历，寻找 name 对应的 password 的值
            if (cursor.moveToNext()) {
                @SuppressWarnings("Range") String psw_query =
cursor.getString(cursor.getColumnIndex("password"));
                //用户名对应的密码与输入的密码相同时
                if (psw_str.equals(psw_query)) {
                    helper.insert(sqLiteDatabase, name_str, paswd_str,
time_str);

                    Toast.makeText(this, "登录成功!",
Toast.LENGTH_SHORT).show();
                    //跳转到 successActivity 页面
                    Intent intent = new Intent(MainActivity.this,
successActivity.class);

                    //intent 会携带上 mstr 的值并以 username 命名
                    intent.putExtra("username", mstr);
                    //开始跳转事件
                    startActivity(intent);
                }
                //密码输入错误时
            } else {
                //跳转到 FaultActivity 页面
                Intent intent2 = new Intent(MainActivity.this,
FaultActivity.class);

                startActivity(intent2);
            }
        }
        //遍历完后发现在表中找不到游标携带的 name 的值时
    } else {
        Toast.makeText(this, "账号不存在，请先注册！",
Toast.LENGTH_SHORT).show();
    }
}

```

```
    }  
} else if (a == 3){  
    //重置按钮功能  
    //将 EditText 的文本清空  
    username.setText("");  
    password.setText("");  
}  
}  
}
```